🧠 MindMate – Local AI Chatbot with Persistent Memory

MindMate is a locally running AI chatbot that can understand conversations, remember previous interactions, and provide context-aware responses. It uses **Ollama** to run a local Large Language Model (LLM), **LangChain** to manage conversational memory, and **SQLite** to store chat history permanently.

The application is developed using **Python and Streamlit**, providing a simple and interactive chatbot interface.

Features

- **Local AI Processing:** Runs the Qwen 2.5 3B model locally using Ollama.
- **Conversational Memory:** Uses LangChain's ConversationBufferMemory to maintain conversation context.
- **Persistent Chat History:** Stores conversations and messages in an SQLite database.
- **Context-Aware Responses:** Remembers previous messages and uses them to answer follow-up questions.
- **Multiple Conversations:** Allows users to create and switch between different conversations.
- **Clear Conversation:** Allows users to clear the messages in the current conversation.
- **Privacy-Friendly:** Conversations are processed locally through Ollama rather than requiring a cloud-based LLM API.

Technologies Used

Technology	**Purpose**
Python	Core programming language
Streamlit	Interactive chatbot interface
Ollama	Runs the local language model
Qwen 2.5 3B	Language model for generating responses

LangChain	Manages conversation flow and memory
ConversationBufferMemory	Maintains conversational context
SQLite	Stores chat history persistently

System Architecture

The application follows this workflow:

1. The user enters a message through the Streamlit interface.
2. The message is passed to the LangChain ConversationChain.
3. ConversationBufferMemory provides the previous conversation context.
4. The prompt and conversation history are sent to the local Qwen model through Ollama.
5. Ollama generates a response.
6. The response is displayed in the Streamlit interface.
7. Both the user message and AI response are saved in the SQLite database.
8. When an existing conversation is reopened, its stored messages are loaded back into conversational memory.

Project Structure

MindMate_Local_AI_Chatbot/

```
│
├── app.py            # Main Streamlit application
├── mindmate.db       # SQLite database (created automatically)
└── README.md         # Project documentation
```

3. Install Required Packages

pip install -U streamlit langchain-classic langchain-ollama

4. Install Ollama

Download and install Ollama from:

https://ollama.com/

5. Download the Qwen Model

Open your terminal and execute:

ollama pull qwen2.5:3b

6. Run the Application

streamlit run app.py

The application will open in your default browser.

Example Interaction

User: My name is Subashini.

MindMate: Nice to meet you, Subashini!

User: I am a Computer Science teacher.

MindMate: That's great! I can help you with teaching and programming-related tasks.

User: What is my name?

MindMate: Your name is Subashini.

User: What is my profession?

MindMate: You are a Computer Science teacher.

This demonstrates how MindMate uses previous conversation context to answer follow-up questions.

How Memory Works

MindMate uses two layers of memory:

1. ConversationBufferMemory – Short-Term Conversation Context

LangChain's ConversationBufferMemory stores the conversation history during an active conversation.

It helps the language model understand previous messages and maintain conversational continuity.

2. SQLite – Persistent Conversation Storage

SQLite stores the conversation details in a local database.

The database maintains two tables:

Conversations Table

Column	**Description**
id	Unique conversation ID
title	Conversation title
created_at	Conversation creation time
updated_at	Last updated time

Messages Table

Column	**Description**
id	Unique message ID
conversation_id	Associated conversation ID
role	User or assistant
content	Message content
created_at	Message timestamp

When a conversation is reopened, the application retrieves its messages from SQLite and restores them into LangChain memory.

Privacy and Data Storage

- The language model runs locally through Ollama.
- No external LLM API key is required.
- Chat history is stored in the local SQLite database.
- The database is maintained within the project directory.

- Users can clear the current conversation through the application.

Note: Local processing does not automatically guarantee complete privacy. Protect the local database and computer appropriately.

Learning Outcomes

This project demonstrates the practical implementation of:

- Large Language Models (LLMs)
- Local AI model execution
- LangChain conversation management
- Conversational memory
- SQLite database integration
- Context-aware chatbot development
- Streamlit application development

Future Enhancements

- Chat export in PDF and text formats.
- Custom system prompts and chatbot personalities.
- Document-based question answering using Retrieval-Augmented Generation (RAG).
- Conversation summarization for long chat histories.
- Search functionality across previous conversations.
- Improved memory management.